

Occ Spec - Web Dev Guide:

Flask Resources	3
HTML	4
Basic HTML Document	4
Attributes	4
Formatting Text	5
Images	6
Tables	6
Lists	7
Text Links	7
CSS	8
Selectors	8
Measurement Units	8
Colours	8
Backgrounds	9
Fonts	9
Text	9
Images	9
Links	9
Tables	10
Borders	10
Margin	10
Lists	10
JavaScript	12
Resources	12
Flask - Routes	13
Flask - Rendering Templates	14
Flask - Jinja Blocks	15
Jinja Template Variables	15

Jinja Template if Statements	15
Jinja Template for Loop	15
Jinja Template Inheritance	16
Jinja Template url for Function	16
Flask - Message Flashing	17
Flask - Web Forms with Flask-WTF	19
Flask - Databases with Flask-SQLAlchemy	22
Setting up SQLAlchemy	22
Creating Models	22
Creating the Database on Run	23
Checking and Added to SQLAlchemy Database	23
Flask - Password Hashing with Flask-Bcrypt	25
Flask - User Logins with Flask-Login	27
Flask - Admin Pages with Flask-Admin	32
Initialisation	32
Adding Model Views	32
Adding Custom Model Views	32
Flask - Web Page Error Handling	34
Bootstrap	35
Loading Bootstrap within a Flask Project	35
Using BootStrap	35
OOP	37
Classes and Objects	37
Encapsulation	37
Inheritance	37
Polymorphism	37
Abstraction	37
Method Overloading	38
Method Overriding	38

Flask Resources

The Flask Mega-Tutorial (<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-i-hello-world>)

Flask Documentation (<https://flask.palletsprojects.com/en/stable/>)

How to Make a Web App Using Flask (<https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3>)

Flask Tutorial (<https://www.geeksforgeeks.org/flask-tutorial/?ref=lbp>)

HTML

More Info:

<https://www.w3schools.com/html/default.asp>

HTML and CSS Online Course (<https://www.freecodecamp.org/learn/2022/responsive-web-design/>)

Basic HTML Document

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title> </title>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="styles.css" />
  <!-- Adding a comment -->
</head>
<body>
<main> (specifies main or important content in the document)
</main>
</body>
</html>
```

Heading Tag:

<h1>Biggest heading</h1>
to
<h6>Smallest heading</h6>

Paragraph Tag:

<p> </p>

Line Break Tag:

Centre Tag:

<centre><p> </p></centre>

Horizontal Line Tag:

<hr />

Preserve Formatting Tag:

<pre> </pre>

Mobile responsiveness:

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

Attributes

More Info:

https://www.w3schools.com/html/html_attributes.asp

Core Attributes:

id="x" Can be used to uniquely identify any element within an HTML page.

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

<h1>Heading 1</h1>

<h2>Heading 2</h2>

<h3>Heading 3</h3>

<h4>Heading 4</h4>

<h5>Heading 5</h5>

<h6>Heading 6</h6>

title="x" Gives a suggested title for the element and is often displayed as a tooltip when cursor come over the element.
class="x y z" Is used to associate an element with a style sheet and specifies the class of element
style="x: y; z:a;" Allows you to specify CSS rules within the element

International Writing:

dir="x" with x being either "ltr" (left to right) or "rtl" (right to left) allows you to indicate to the browser about the direction of text.

lang="en" allows you to indicate the main language used in a document, it uses the ISO-639 standard two-character language codes.

Generic Attributes:

Attribute:	Options:	Function:
Align	Right, left, centre	Horizontally aligns tags
Valign	Top, middle, bottom	Vertically aligns tags
Bgcolor	Numeric, hex, RGB value	Place a background colour
Background	URL	Place a background image
Width	Numeric Value	
height	Numeric Value	

Formatting Text

More Info:

https://www.w3schools.com/html/html_formatting.asp

Bold Text: ...

Italic Text: <i>...</i>

Underline Text: <u>...</u>

Strike Text: <strike>...</strike>

Monospaced Text: <tt>...</tt>

Superscript Text: ^{...}

Subscript Text: _{...}

Larger Text: <big>...</big>

Smaller Text: <small>...</small>

Grouping Content (Old Way):

<div>...</div> or ...

<i>Italic</i>	<i>Italic</i>
Bold	Bold
Emphasized	<i>Emphasized</i>
Strong	Strong
small	small
Deleted	Deleted
<ins>Inserted</ins>	<u>Inserted</u>
v_f	v _f
a²	a ²
<mark>Marked</mark>	Marked

Semantic HTML:

More Info:

https://www.w3schools.com/html/html5_semantic_elements.asp

A semantic element clearly describes its meaning to both the browser and the developer.

Examples of **non-semantic** elements: <div> and - Tells nothing about its content.

Examples of **semantic** elements: <form>, <table>, and <article> - Clearly defines its content.

In HTML there are some semantic elements that can be used to define different parts of a web page:

- <article>
- <aside>
- <details>

- <figcaption>
- <figure>
- <footer>
- <header>
- <main>
- <mark>
- <nav>
- <section>
- <summary>
- <time>

Images

More Info:

https://www.w3schools.com/html/html_images.asp

Insert Image: or

Additional image attributes:

Width=""

Height=""

Border=""

Align=""

Tables

More Info:

https://www.w3schools.com/html/html_tables.asp

<table>

<tr> (creates table rows)

<th>Heading 1</th> (creates table heading)

<th>Heading 2</th>

</tr>

<tr>

<td>Row 1, Column 1</td> (create data cells)

<td>Row 1, Column 2</td>

</tr>

<tr>

<td>Row 2, Column 1</td>

<td>Row 2, Column 2</td>

</tr>

</table>

<thead> - to create a separate table header.

<tbody> - to indicate the main body of the table.

<tfoot> - to create a separate table footer.

Additional table attributes:

Cellpadding="" (distance between cell borders and the content within a cell)

Cellspacing="" (defines the width of the border)

Colspan="" (merges two or more columns into a single column)

Rowspan="" (merge two or more rows)

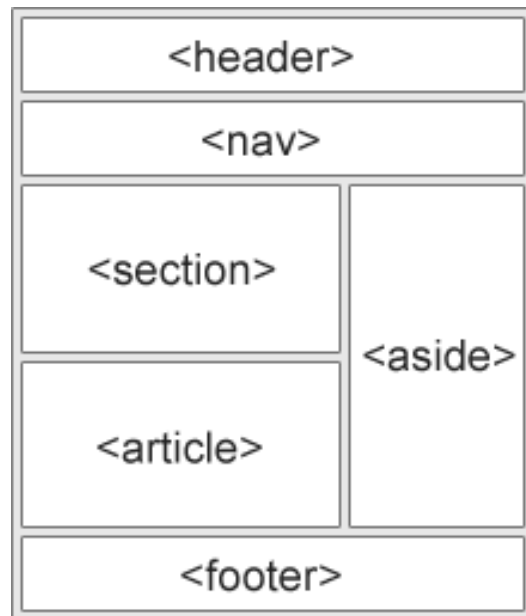
Bgcolor=""

Background=""

Width=""

Height=""

Caption="" (serve as a title or explanation for the table)



Lists

More Info:

https://www.w3schools.com/html/html_lists.asp

- `<ul>` (an unordered list - this will list using plain bullets)
- `<ol>` (an ordered list - use different schemes of numbers to list your items)
- `<dl>` (a definition list - arranges like a dictionary)
- `<li>` lists individual items in a list

Additional list attributes:

`Type=""` (specify the type of bullet you like - square, disc, circle in `<ul>`)

Text Links

More Info:

https://www.w3schools.com/html/html_links.asp

`<a href="URL">...</a>` (anchor tag - anything between the opening and closing tag becomes part of the link)

Additional link attributes:

`Target=""`

`_blank` - opens in a new window or tab

`_self` - open in the same frame

`_parent` - opens in the parent frame

`_top` - opens in the full body of the window

`Targetframe` - opens in a named targetframe

`Link="colour"` - set colour of link

`Alink="colour"` - set colour of active link

`Vlink="colour"` - set colour of visited link

CSS

More Info:

<https://www.w3schools.com/css/default.asp>

Selectors

More Info:

https://www.w3schools.com/css/css_selectors.asp

Type: eg. H1 {} (any element)

Universal: * {}

Descendant: eg. ul em {} (element within another element)

Class: eg. .black {} or h1.black {}

ID: eg. #... {} or h1#... {}

Child: eg. body > p {} (different from descendant as only a direct child element would be effected)

Attribute: eg. input[type="text"] {}

Grouping: eg. h1, #..., .black {}

Measurement Units

More Info:

https://www.w3schools.com/css/css_units.asp

Unit:	Description:
%	A percentage relative to another value, typically an enclosing element
cm	Measurement in centimetres
em	A relative measurement for the height of a font in em spaces.
ex	A measurement relative to a font's x-height
in	Measurement in inches
mm	Measurement in millimetres
pc	A measurement in picas
pt	A measurement in points (defined as 1/72nd of an inch)
px	A measurement in screen pixels

Colours

More Info:

https://www.w3schools.com/css/css_colors.asp

Format:	Syntax:
Hex code	#RRGGBB
Short hex code	#RGB
RGB %	rgb(rrr%, ggg%, bbb%)
RGB absolute	rgb(rrr, ggg, bbb)
Keyword	aqua, black, etc.

Backgrounds

More Info:

https://www.w3schools.com/css/css_background.asp

- Background-color: Set the background colour of an element
- Background-image: Set the background image of an element
- Background-repeat: Controls the repetition of an image in the background
- Background-position: Controls the position of an image in the background
- Background-attachment: Controls the scrolling of an image in the background
- Background: shorthand

Fonts

More Info:

https://www.w3schools.com/css/css_font.asp

- Font-family: Change the face of a font
- Font-style: Make a font italic or oblique
- Font-variant: Creates a small-caps effect
- Font-weight: Increase or decrease how bold or light a font appears
- Font-size: Increase or decrease the size of a font
- Font: shorthand

Text

More Info:

https://www.w3schools.com/css/css_text.asp

- Color: Set the colour of a text
- Direction: Set the text direction
- Letter-spacing: Used to add or subtract space between the letters that make up a word
- Word-spacing: Used to add or subtract space between the words of a sentence
- text-indent: Used to indent the text of a paragraph
- Text-align: Used to align the text of a document
- Text-decoration: Used to underline, over line, and strike through text
- Text-transform: Used to capitalise text to convert text to uppercase or lowercase letters
- White-space: Used to control the flow and formatting of text
- Text-shadow: Used to set the text shadow around a text

Images

More Info:

https://www.w3schools.com/css/css_image_gallery.asp

- Border: Used to set the width of an image border
- Height: Used to set the height of an image
- Width: Used to set the width of an image
- Moz-opacity: Used to set the opacity of an image:

Links

More Info:

https://www.w3schools.com/css/css_link.asp

- :link - Signifies unvisited links
- :visited - Signifies visited links
- :hover - Signifies when a user hovers over a link
- :active - Signifies when a user is currently clicking

Tables

More Info:

https://www.w3schools.com/css/css_table.asp

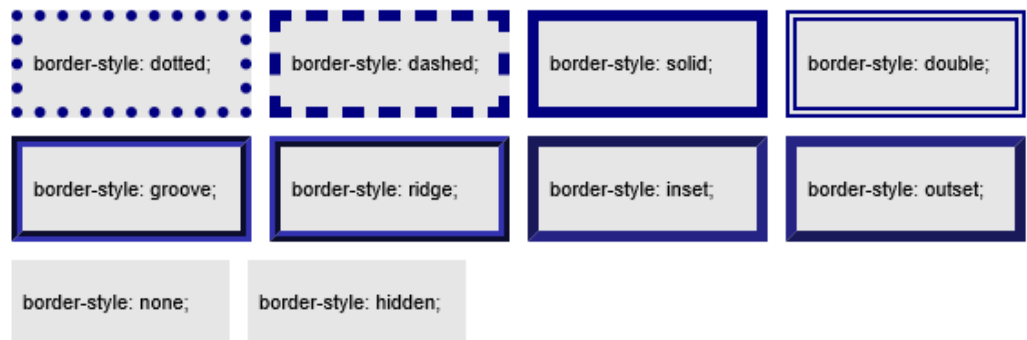
- Border-collapse: Specifies whether the browser should control the appearance of the adjacent border that touch each other or whether each cell should maintain its style
- Border-spacing: Specifies the width that should appear between table cells
- Empty-cells: Specifies whether the border should be shown if a cell is empty
- Table-layout: Allows browsers to speed up the layout of a table by using the first width properties it come across for the rest of a column rather than having to load the whole table before rendering it

Borders

More Info:

https://www.w3schools.com/css/css_border.asp

- Border-color: Specifies the colour of a border
- Border-bottom-color
- Border-top-color
- Border-left-color
- Border-right-color
- Border-style:
 - None
 - Solid
 - Dotted
 - Dashed
 - Double
 - Groove
 - Ridge
 - Inset
 - Outset
 - Hidden
- Border-bottom-style
- Border-top-style
- Border-left-style
- Border-right-style
- Border-width:
 - Border-bottom-width
 - Border-top-width
 - Border-left-width
 - Border-right-width
- Border: shorthand



Margin

More Info:

https://www.w3schools.com/css/css_margin.asp

- Margin-top:
- Margin-bottom:
- Margin-left:
- Margin-right:
- Margin: shorthand

Lists

More Info:

https://www.w3schools.com/css/css_list.asp

- List-style-type:
 - None
 - Disc
 - Circle
 - Square
 - decimal
 - Decimal-leading-zero
 - Lower-alpha and upper-alpha
 - Lower-roman and upper-roman
 - Lower-Greek and upper-Greek
 - Lower-Latin and upper-Latin
 - Hebrew
 - Armenian
 - Georgian
- List-style-position: Specifies whether a long point that wraps to a second line should align with the first line or start underneath the start of the marker
 - None
 - Inside
 - Outside
- List-style-image: Specifies an image for the marker
- List-style: Shorthand
- Marker-offset: Specifies the distance between a marker and the text in the list

<i>armenian</i>	<i>circle</i>	<i>decimal</i>	<i>georgian</i>	<i>decimal-leading-zero</i>	<i>lower-alpha</i>	<i>lower-greek</i>	<i>lower-roman</i>	<i>square</i>	<i>upper-alpha</i>	<i>upper-roman</i>
Ա.	°	1.	Ⴀ.	01.	a.	α.	i.	■	A.	I.
Բ.	°	2.	Ⴁ.	02.	b.	β.	ii.	■	B.	II.
Գ.	°	3.	Ⴂ.	03.	c.	γ.	iii.	■	C.	III.
Դ.	°	4.	Ⴃ.	04.	d.	δ.	iv.	■	D.	IV.
Ե.	°	5.	Ⴄ.	05.	e.	ε.	v.	■	E.	V.

JavaScript

Resources

JavaScript Tutorial by W3Schools (<https://www.w3schools.com/js/default.asp>)

JavaScript Information (<https://javascript.info/>)

JavaScript Online Course (<https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures-v8/>)

Flask - Routes

More Info:

<https://www.geeksforgeeks.org/flask-app-routing/?ref=lbp>

App Routing means mapping the URLs to a specific function that will handle the logic for that URL. To bind a function to an URL path use the app.route decorator.

```
from flask import Flask
app = Flask(__name__)
# Pass the required route to the decorator.
@app.route("/hello")
def hello():
    return "Hello"
```

```
@app.route("/")
def index():
    return "Homepage"
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

Dynamic URLs – You can also build dynamic URLs by using variables in the URL. To add variables to URLs, use <variable_name> rule. The function then receives the <variable_name> as keyword argument.

```
@app.route('/user/<username>')
def show_user(username):
    # Greet the user
    return f'Hello {username} !'
```

Flask - Rendering Templates

More Info:

https://www.geeksforgeeks.org/flask-rendering-templates/?ref=next_article

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-ii-templates>

Create a folder called “templates” in the current folder. Inside this “templates” folder, all of the templates (HTML files) will be residing.

```
@app.route("/")
def index():
    return render_template("index.html")
```

Flask - Jinja Blocks

More Info:

<https://www.geeksforgeeks.org/templating-with-jinja2-in-flask/>

Jinja Template Variables

To declare the variable using Jinja Template you use `{{variable_name}}` within the HTML file. As a result, the variable will be displayed on the Website.

```
<html>
  <head>
    <title>Variable Example</title>
  </head>
  <body>
    <h3>Hello {{name}}</h3>
  </body>
</html>
```

```
@app.route("/")
def index():
    return render_template("index.html", name=name)
```

Jinja Template if Statements

Just like declaring the variable in the Jinja template, if conditions have almost similar syntax. But here we specify the beginning and end of the if block.

```
<html>
  <head>
    <title>If example</title>
  </head>
  <body>
    {% if(name == "This Name") %}
      <h3> Welcome </h3>
    {% else %}
      <h3> Unknown name entered: {{name}} </h3>
    {% endif %}
  </body>
</html>
```

Jinja Template for Loop

Jinja for loop syntax is similar to the if statements, the only difference is for loop requires sequence to loop through.

```
<html>
  <head>
    <title>For Loop Example</title>
  </head>
  <body>
    <h2> Available Course </h2>
    {% for course in courses%}
      <h4> {{course}} </h4>
    {% endfor %}
  </body>
</html>
```

Jinja Template Inheritance

Syntax of Jinja Template inheritance:

```
base.html
{% block content %}
{% endblock %}

index.html
{% extends "base.html" %}
    {% block content %}
        ....
    {% endblock %}
```

Jinja Template url_for Function

To build a dynamic website you need multiple re-direction within the website. url_for function is a handy method that helps in re-direction from one page to another. url_for is also used to link HTML templates with static CSS or JavaScript files.

```
@app.route("/choice/<pick>")
def choice(pick):
    if pick == 'variable':
        return redirect(url_for('var'))
    if pick == 'if':
        return redirect(url_for('if_else'))
    if pick == 'for':
        return redirect(url_for('for_loop'))
```


Flask - Message Flashing

More Info:

<https://www.geeksforgeeks.org/flask-message-flashing/>

<https://flask.palletsprojects.com/en/stable/patterns/flashing/>

Message Flashing means when you click on any button and the website could immediately flash any message that can be an error, alert, or any type of message, is call message Flashing and when that happens in a flask or you are creating any website which shows an alert, error type message than its call Flask-Message Flashing.

A Flask contains a flash() function. which passes a request to the next user. which generally is a template you are using flash methods in the above template in which the message parameter is the real message to be flashed, and another one is the optional category parameter category, it will happen "error" or "warning".

```
@app.route("/login", methods=['GET', 'POST'])
def login():
    error = None
    if request.method == "POST":
        if request.form['pass'] != "GFG":
            error = "Invalid Password"
        else:
            flash("You are successfully login into the Flask
Application")
            return redirect(url_for('row'))
    return render_template("login.html", error=error)
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Home</title>
</head>
<body>
{% with messages = get_flashed_messages()%}
```

```
{% if messages%}
    {% for message in messages%}
        <p>{{message}}</p>
    {%endfor%}
{%endif%}
{%endwith%}
<h3>Welcome</h3>
<a href="{{url_for('login')}}">login</a>
</body>
</html>
```

Flask - Web Forms with Flask-WTF

More Info:

<https://www.geeksforgeeks.org/create-contact-us-using-wtforms-in-flask/>

<https://flask-wtf.readthedocs.io/en/1.2.x/>

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-iii-web-forms>

<https://www.digitalocean.com/community/tutorials/how-to-use-and-validate-web-forms-with-flask-wtf>

WTForms is a library designed to make the processing of forms easier to manage. It handles the data submitted by the browser very easily.

Advantages of WT-FORM:

- Don't have to worry about validators.
- Avoidance of Cross-Site Request Forgery (CSRF).
- WTForms come as classes, so all the good come's from an object form.
- No need to create any <label> or <input> elements manually using HTML.

Step 1: Create a class having all elements that you want in your Form

```
from flask_wtf import FlaskForm
from wtforms import StringField, validators, PasswordField, SubmitField
from wtforms.validators import DataRequired, Email
class contactForm(FlaskForm):
    name = StringField(label='Name', validators=[DataRequired()])
    email = StringField(label='Email', validators=[
        DataRequired(), Email(granular_message=True)])
    message = StringField(label='Message')
    submit = SubmitField(label="Log In")
```

Step 2: Create the object of the form and pass the object as a parameter in the render_template

```
@app.route("/", methods=["GET", "POST"])
def home():
    form = contactForm()
    return render_template("contact.html", form=form)
```

Step 3: Add CSRF protection. And add a secret key.

```
app.secret_key = "any-string-you-want-just-keep-it-secret"
```

{{ form.csrf_token }} is used to provide csrf protection.

```
<!DOCTYPE HTML>
```

```

<html>
  <head>
    <title>Contact</title>
  </head>
  <body>
    <div class="container">
      <h1>Contact Us</h1>
      <form method="POST" action="{{ url_for('home') }}">
        {{ form.csrf_token }}
        <p>
          {{ form.name.label }}
          <br>
          {{ form.name }}
        </p>
        <p>
          {{ form.email.label }}
          <br>
          {{ form.email(size=30) }}
        </p>
        <p>
          {{ form.message.label }}
          <br>
          {{ form.message }}
        </p>
        {{ form.submit }}
      </form>
    </div>
  </body>
</html>

```

Step 5: Validating the Form and receiving the data.

```

@app.route("/", methods= ["GET", "POST"])
def home():
    form = contactForm()
    if form.validate_on_submit():
        print(f"Name:{form.name.data}",
              f"E-mail: {form.email.data}",
              f"message: {form.message.data}")
    else:
        print("Invalid Credentials")
    return render_template("contact.html", form=form)

```

Step 6: Retrieving data from a Form

Format:

`form_name.column_name.data`

Example (It uses a class 'CurrentUser' to store the current user):

```

@main.route("/signup", methods= ["GET", "POST"])
def signup():
    form = SignupForm()
    if form.validate_on_submit():
        new_user = CurrentUser(form.email.data,
                               form.password.data,
                               form.name.data)
        if new_user.check_for_dup_emails():
            flash("Email address is already used!", "info")
            return redirect("/signup")
        else:
            new_user.add_user_to_db()
            return redirect("/login")
    return render_template("signup.html", form=form)

```

Flask - Databases with Flask-SQLAlchemy

More Info:

<https://www.geeksforgeeks.org/connect-flask-to-a-database-with-flask-sqlalchemy/>

<https://flask-sqlalchemy.palletsprojects.com/en/stable/>

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-iv-database>

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-viii-followers>

Setting up SQLAlchemy

```
from flask import Flask

from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

# adding configuration for using a sqlite database

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'

# Creating an SQLAlchemy instance

db = SQLAlchemy(app)
```

Creating Models

In SQLAlchemy you use classes to create our database structure. For this example, it creates a Profile table that will be responsible for holding the user's id, first name, last name, and age.

```
# Models

class Profile(db.Model):

    # Id : Field which stores unique id for every row in
    # database table.

    # first_name: Used to store the first name if the user

    # last_name: Used to store last name of the user

    # Age: Used to store the age of the user

    id = db.Column(db.Integer, primary_key=True)

    first_name = db.Column(db.String(20), unique=False, nullable=False)

    last_name = db.Column(db.String(20), unique=False, nullable=False)

    age = db.Column(db.Integer, nullable=False)

    # repr method represents how one object of this datatable
    # will look like

    def __repr__(self):
```

```
return f"Name : {self.first_name}, Age: {self.age}"
```

Keyword	What it does
Column	used to create a new column in the database table
Integer	An integer data field
primary_key	If set to True for a field ensures that the field can be used to uniquely identify objects of the data table.
String	An string data field. String(<maximum length>)
unique	If set to True it ensures that every data in that field is unique.
nullable	If set to False it ensures that the data in the field cannot be null.
__repr__	Function used to represent objects of the data table.

Creating the Database on Run

```
if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    app.run()
```

Checking and Added to SQLAlchemy Database

Format:

```
DataBase_Name.query.filter_by(column_name=data).first()
```

Example:

```
def check_for_dup_emails(email):
    is_existing_email = DataBase_Name.query.filter_by(email=email.email).first()
    return is_existing_email
```

Format:

```
new_record_name = DataBase_Name(column_name=data, ...)
```

Example:

```
def add_user_to_db(email, password, name):  
    new_record = DataBase_Name(email=email, password=password, name=name)  
    db.session.add(new_record)  
    db.session.commit()
```


Flask - Password Hashing with Flask-Bcrypt

More Info:

<https://www.geeksforgeeks.org/password-hashing-with-bcrypt-in-flask/>

<https://flask-bcrypt.readthedocs.io/en/1.0.1/>

Bcrypt is a popular hashing algorithm used to hash passwords. It is a password-hashing function that is based on the Blowfish cipher and is designed to be slow and computationally expensive, making it more difficult for attackers to guess or crack passwords.

Key Terminologies:

- Password Hashing: The process of converting a plaintext password into a hashed or encrypted format.
- Bcrypt: A password-hashing function based on the Blowfish cipher.
- Salt: Random data that is used as additional input to a one-way function that hashes a password or passphrase.
- Hashing Algorithm: A mathematical function that converts a plaintext password into a fixed-length hash value.
- Iterations: The number of times a password is hashed using the bcrypt algorithm.

Step 1: Import Flask-Bcrypt

```
from flask_bcrypt import Bcrypt
```

Step 2: Create a Bcrypt Object

```
bcrypt = Bcrypt(app)
```

Step 3: Hash a Password

```
hashed_password = bcrypt.generate_password_hash('password').decode('utf-8')
```

Step 4: Verify a Password

The `check_password_hash()` function returns True if the password matches the hashed password, otherwise, it returns False.

```
is_valid = bcrypt.check_password_hash(hashed_password, 'password')
```

Example of Hashing a Password within a larger project:

```
@main.route("/signup", methods=["GET", "POST"])
def signup():
    form = SignupForm()
    if form.validate_on_submit():
        new_user = CurrentUser(form.email.data,
                                bcrypt.hashpw(form.password.data.encode("utf-8"), bcrypt.gensalt()),
                                form.name.data)
    ...
```

Example of Checking a Password within a larger project:

```
@main.route("/login", methods=["GET", "POST"])
def login():
    form = LoginForm()
    if form.validate_on_submit():
        current_user = UserDB.query.filter_by(email=form.email.data).first()
        if not current_user:
            flash("Incorrect Email!", "warning")
            return redirect("/login")
        if not check_password_hash(current_user.password, form.password.data):
            flash("Incorrect Password!", "warning")
            return redirect("/login")
```

Flask - User Logins with Flask-Login

More Info:

<https://www.geeksforgeeks.org/how-to-add-authentication-to-your-app-with-flask-login/>

<https://www.geeksforgeeks.org/flask-role-based-access-control/>

<https://flask-login.readthedocs.io/en/latest/>

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-v-user-logins>

<https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>

Flask-Login provides a simplified way of managing users, which includes easily logging in and out users, as well as restricting certain pages to authenticated users. It uses Flask-SQLAlchemy.

Step 1: Import the necessary modules.

You first import the classes you need from Flask, Flask-SQLAlchemy, and Flask-Login. You then create a flask application, indicate what database Flask-SQLAlchemy should connect to, and initialise the Flask-SQLAlchemy extension. You also need to specify a secret key, which can be any random string of characters, and is necessary as Flask-Login requires it to sign session cookies for protection against data tampering. Next, you initialise the LoginManager class from Flask-Login, to be able to log in and out users.

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
# Create a flask application
app = Flask(__name__)
# Tells flask-sqlalchemy what database to connect to
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///db.sqlite"
# Enter a secret key
app.config["SECRET_KEY"] = "ENTER YOUR SECRET KEY"
# Initialise flask-sqlalchemy extension
db = SQLAlchemy()
# LoginManager is needed for our application
# to be able to log in and out users
login_manager = LoginManager()
login_manager.init_app(app)
```

Step 2: Create a User Model & Database

To be able to store users' information such as their username and password, you need to create a table with Flask-SQLAlchemy, this is done by creating a model that represents the information you want to store. In this case, you first create a Users class and make it a subclass of db.Model to make it a model with the help of Flask-SQLAlchemy. You also make the Users class a subclass of UserMixin, which will help to implement properties such as is_authenticated to the Users class. You will also need to create columns within the user model, to store individual attributes, such as

the user's username. When creating a new column, you need to specify the datatype such as `db.Integer` and `db.String` as well. When creating columns, you also need to specify keywords such as `unique = True`, if you want to ensure values in the column are unique, `nullable = False`, which indicates that the column's values cannot be NULL, and `primary_key = True`, which indicates that the row can be identified by that `primary_key` index. Next, the `db.create_all` method is used to create the table schema in the database.

```
# Create user model
class Users(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(250), unique=True,
                        nullable=False)
    password = db.Column(db.String(250),
                        nullable=False)

# Initialize app with extension
db.init_app(app)

# Create database within app context
with app.app_context():
    db.create_all()
```

Step 3: Adding a user loader

Before implementing the functionality for authenticating the user, you need to specify a function that Flask-Login can use to retrieve a user object given a user id. This functionality is already implemented by Flask-SQLAlchemy, you simply need to query and use the `get` method with the user id as the argument.

```
# Creates a user loader callback that returns the user object given an id
@login_manager.user_loader
def loader_user(user_id):
    return Users.query.get(user_id)
```

Step 4: Registering new accounts with Flask-Login

To allow the user to register an account, you need to create the HTML template. This will need to contain a form that allows the user to enter their details, such as their username and chosen password. Then use the following code to create an instance of the user class and add it to the database.

```
@app.route('/register', methods=["GET", "POST"])
def register():
```

```

# If the user made a POST request, create a new user
if request.method == "POST":
    user = Users(username=request.form.get("username"),
                  password=request.form.get("password"))

    # Add the user to the database
    db.session.add(user)

    # Commit the changes made
    db.session.commit()

    # Once user account created, redirect them
    # to login route (created later on)
    return redirect(url_for("login"))

# Renders sign_up template if user made a GET request
return render_template("sign_up.html")

```

Step 5: Allowing users to log in with Flask-Login

Like with creating the registered route, you first need a way for the user to log in through an HTML form. With the login route, you do the same thing of checking if the user made a POST request. If they did, you filter the users within the database for a user with the same username as the one being submitted. Next, you check if that user has the same password as the password the user entered in the form. If they are the same, you log-in to the user by using the `login_user` function provided by Flask-Login. You can then redirect the user back to a route with a function called "home". If the user didn't make a POST request, and instead a GET request, then you'll render the login template.

```

@app.route("/login", methods=["GET", "POST"])
def login():
    # If a post request was made, find the user by
    # filtering for the username
    if request.method == "POST":
        user = Users.query.filter_by(
            username=request.form.get("username")).first()

        # Check if the password entered is the
        # same as the user's password
        if user.password == request.form.get("password"):
            # Use the login_user method to log in the user
            login_user(user)

```

```

        return redirect(url_for("home"))

    # Redirect the user back to the home page

    # (we'll create the home route in a moment)

    return render_template("login.html")

```

Step 6: Conditionally rendering HTML based on the user's authentication status with Flask-Login

When using Flask, it uses Jinja to parse the templates. Jinja is a templating engine that allows us to add code, such as if-else statements within our HTML, you can then use it to conditionally render certain elements depending on the user's authentication status for example the `current_user` variable is exported by Flask-Login, and you can use it within the Jinja template to conditionally render HTML based on the user's authentication status.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Home</title>
  </head>
  <body>
    <nav>
      <ul>
        <li><a href="/login">Login</a></li>
        <li><a href="/register">Create account</a></li>
      </ul>
    </nav>
    {% if current_user.is_authenticated %}
    <h1>You are logged in</h1>
    {% else %}
    <h1>You are not logged in</h1>
    {% endif %}
  </body>
</html>

```

OR

```
@main.route("/user", methods=["GET", "POST"])
@login_required
def user():
    flash(f"Welcome {flask_login.current_user.name}!")
    return render_template("user.html")
```

OR you can limit access based on a user role

```
@main.route("/staff")
@login_required
def staff_home():
    if current_user.access_level != "S":
        flash(f"You must be a staff member!")
        return redirect("/")
    else:
        flash(f"Welcome {current_user.name}!")
        return render_template("staff/staffHome.html")
```

Step 7: Adding Logout Functionality

```
@main.route("/logout")
@login_required
def logout():
    flash("Goodbye!")
    logout_user()
    return redirect("/")
```

Flask - Admin Pages with Flask-Admin

More Info:

<https://www.geeksforgeeks.org/how-to-integrate-flask-admin-and-flask-login/>

<https://flask-admin.readthedocs.io/en/latest/>

<https://flask-admin.readthedocs.io/en/latest/introduction/#getting-started>

Flask-Admin provides an easy-to-use layer on top of a number of databases and file stores.

Initialisation

The first step is to initialise an empty admin interface for your Flask app:

```
from flask import Flask
from flask_admin import Admin
app = Flask(__name__)
admin = Admin(app, name='Admin Pages',
theme=Bootstrap4Theme(swatch='cerulean'))
```

If you start this application and navigate to <http://localhost:5000/admin/>, you should see an empty page with a navigation bar on top. Customise the look by specifying one of the included BootSwatch themes (see <https://bootswatch.com/4/> for a preview of the swatches).

Adding Model Views

Model views allow you to add a dedicated set of admin pages for managing any model in your database. Do this by creating instances of the *ModelView* class, which you can import from one of Flask-Admin's built-in ORM backends.

```
from flask_admin.contrib.sqla import ModelView
# Flask and Flask-SQLAlchemy initialisation here
admin = Admin(app, name='Admin Pages', theme=Bootstrap4Theme())
admin.add_view(ModelView(User, db.session))
admin.add_view(ModelView(Post, db.session))
```

Adding Custom Model Views

```
class UserDBView(ModelView):
    def is_accessible(self):
        return current_user.is_authenticated and current_user.access_level == "1"

    can_edit = False
    can_create = False
    can_delete = True
```



```
can_export = True

column_list = ("email", "name", "access_level")
column_searchable_list = ("email", "name")
column_filters = ("email", "name")
```

```
if __name__ == "__main__":
    with app.app_context():
        db.create_all()
        admin.add_view(UserDBView(UserDB, db.session))
        admin.add_view(AddStaff(name="Add Staff Member"))
        admin.add_view(LogOut(name="Log Out"))
    app.run()
```

Flask - Web Page Error Handling

More Info:

<https://www.geeksforgeeks.org/python-404-error-handling-in-flask/>

<https://flask.palletsprojects.com/en/stable/errorhandling/>

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-vii-error-handling>

```
from flask import Flask, render_template

app = Flask(__name__)

# app name

@app.errorhandler(404)

# inbuilt function which takes error as parameter

def not_found(e):

    # defining function

    return render_template("404.html")
```

The above python program will return 404.html file whenever the user opens a broken link.

Bootstrap

More Info:

<https://www.w3schools.com/bootstrap5/>

https://www.w3schools.com/bootstrap/bootstrap_ref_all_classes.asp

<https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial-part-xi-facelift>

One of the most popular CSS frameworks is Bootstrap. If you want to see the kind of pages that can be designed with this framework, the documentation has some [examples](#).

These are some benefits you get when using Bootstrap to style your web pages:

- Similar look in all major web browsers
- Handling of desktop, tablet and phone screen sizes
- Customisable layouts
- Nicely styled navigation bars, forms, buttons, alerts, popups, etc.

Loading Bootstrap within a Flask Project

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Log In System</title>
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/css/
bootstrap.min.css" rel="stylesheet">
</head>
<body>
    {% block content %}{% endblock %}
<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/
bootstrap.bundle.min.js"></script>
</body>
</html>
```

Using Bootstrap

Apply styles from Bootstrap's CSS, apply any to the follow as a class attribute to your HTML.

Bootstrap CSS Typography Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_css_text.asp

Bootstrap CSS Buttons Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_css_buttons.asp

Bootstrap CSS Forms Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_css_forms.asp

Bootstrap CSS Helpers Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_css_helpers.asp

Bootstrap CSS Images Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_css_images.asp

Bootstrap CSS Tables Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_css_tables.asp

Bootstrap CSS Drop downs Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_comp_dropdowns.asp

Bootstrap CSS Nav Bars Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_comp_navs.asp

Bootstrap Glyphicons Reference

https://www.w3schools.com/bootstrap/bootstrap_ref_comp_glyphs.asp

OOP

More Info:

<https://www.freecodecamp.org/news/object-oriented-programming-in-python/>

https://www.w3schools.com/python/python_classes.asp

<https://www.geeksforgeeks.org/python-oops-concepts/>

Classes and Objects

Python, like every other object-oriented language, allows you to define classes to create objects. In-built Python classes are the most common data types in Python, such as strings, lists, dictionaries, and so on.

A class is a collection of instance variables and related methods that define a particular object type. You can think of a class as an object's blueprint or template. Attributes are the names given to the variables that make up a class.

A class instance with a defined set of properties is called an object. As a result, the same class can be used to construct as many objects as needed.

Encapsulation

Encapsulation is the process of preventing clients from accessing certain properties, which can only be accessed through specific methods.

Private attributes are inaccessible attributes, and information hiding is the process of making particular attributes private. You use two underscores to declare private characteristics.

Inheritance

Inheritance is regarded as the most significant characteristics of OOP. A class's ability to inherit methods and/or characteristics from another class is known as inheritance.

The subclass or child class is the class that inherits. The superclass or parent class is the class from which methods and/or attributes are inherited.

Polymorphism

The term 'polymorphism' comes from the Greek language and means 'something that takes on multiple forms.'

Polymorphism refers to a subclass's ability to adapt a method that already exists in its superclass to meet its needs. To put it another way, a subclass can use a method from its superclass as is or modify it as needed.

Abstraction

Abstraction isn't supported directly in Python. Calling a magic method, on the other hand, allows for abstraction.

If an abstract method is declared in a superclass, subclasses that inherit from the superclass must have their own implementation of the method.

A superclass's abstract method will never be called by its subclasses. But the abstraction aids in the maintenance of a similar structure across all subclasses.

Method Overloading

The concept of method overloading is found in almost every well-known programming language that follows object-oriented programming concepts. It simply refers to the use of many methods with the same name that take various numbers of arguments within a single class.

Method Overriding

When a method with the same name and arguments is used in both a derived class and a base or super class, we say that the derived class method "overrides" the method provided in the base class.

When the overridden method gets called, the derived class's method is always invoked. The method that was used in the base class is now hidden.